Battle Tanks on FPGA: Overview + Scope

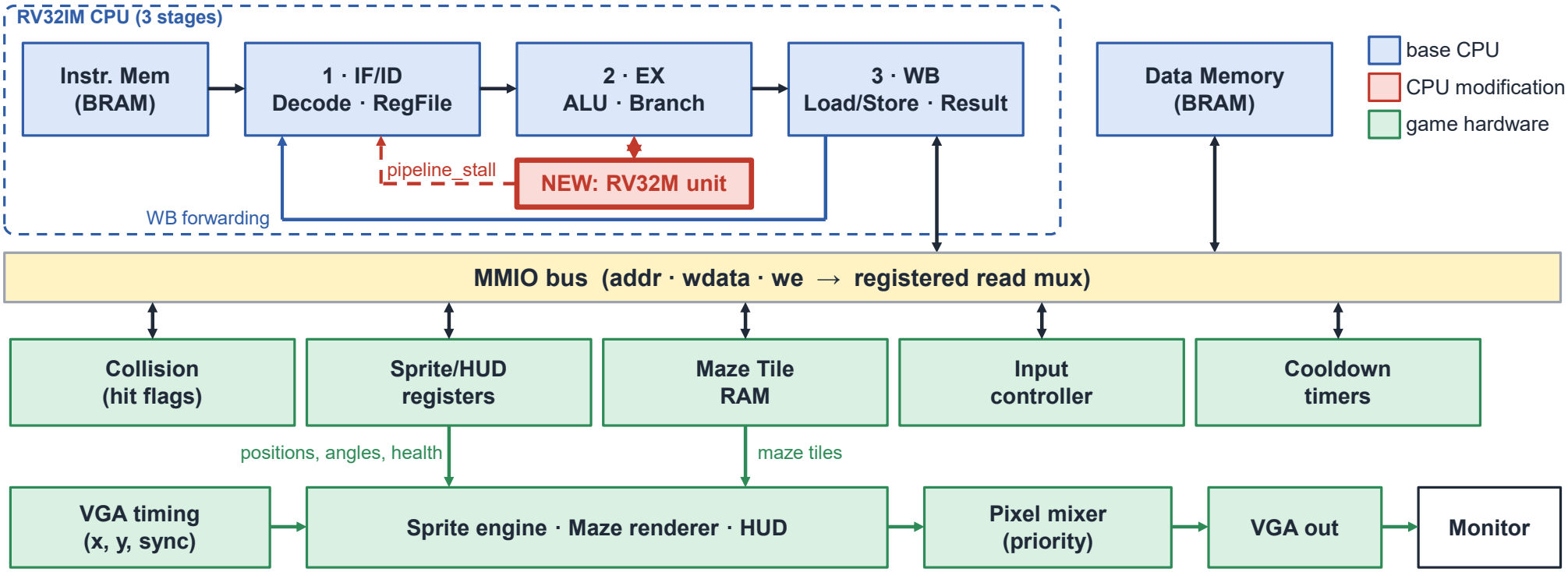
Problem: RV32I has no multiply/divide, so game math is slow.

System: Two-player tank game on a Nexys A7. A 3-stage RV32IM CPU runs game rules in C; hardware handles buttons, timers, collisions and VGA. Our modification: RV32M unit (MUL/DIV/REM) with pipeline stall.

System Scope

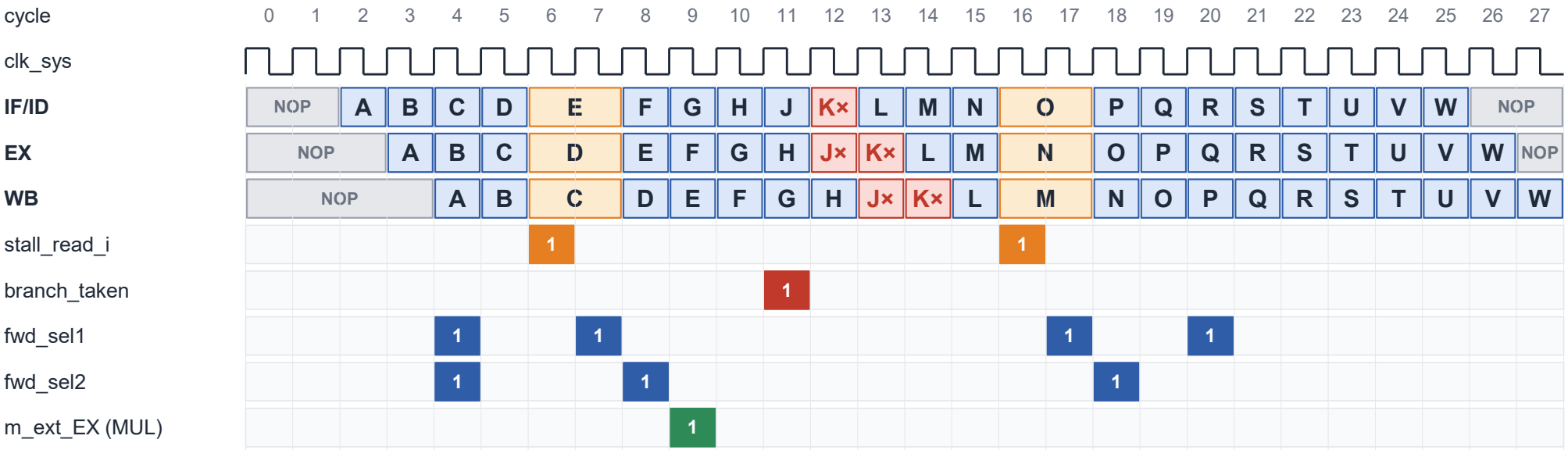
Commitment	• RV32M unit (MUL/DIV/REM) verified and integrated with stall • C program on CPU moves two tanks on a VGA maze using buttons
Goal	• Bullets with wall reflection, collision flags, cooldowns • Health hearts on screen; damage logic in C; full FPGA demo
Stretch	• Single-player mode: software-controlled enemy tank

System Architecture



- **Modification:** RV32M unit plugs into the EX stage; ALU gets its result
- **Control:** divider asserts pipeline_stall; PC and IF/ID freeze
- **Data:** CPU writes sprite regs over MMIO; video reads them every pixel

Timing Diagram: 28 cycles, instruction sequence



Instructions: A addi · B add · C lw · D add · E sw · F mul · G bne (not taken) · H beq (taken) · J, K wrong path · L sw · M lw · N addi (uses M) | orange = held/stalled, red = squashed

- **Forwarding:** B takes A's result in cycle 4; D takes lw data in cycle 7
- **Load stall:** lw waits 1 cycle (6, 16); pipeline holds, single write
- **Taken branch H (11):** J, K squashed; fetch resumes at target L
- **Multi-cycle:** MUL 1 clock; DIV/REM stalls pipeline ≈33 clocks

Design + Verification Plan

Implementation Plan

1. MUL unit → mul.v + testbench

2. DIV/REM unit → div_rem.v + testbench

3. RV32M in pipeline (stall, bubble) → integrated simulation

4. MMIO + VGA + game.c → full simulation, then FPGA

Verification Plan

- **Module level:** self-checking testbenches vs. golden results
- **Edge cases:** signed/unsigned MULH variants, ÷0, INT_MIN ÷ −1, REM sign
- **Processor level:** C programs in simulation: mul/div test, then game.c

Timeline

When	Milestone (concrete deliverables)
By 25 Oct	mul.v + div_rem.v pass their testbenches (incl. ÷0, INT_MIN ÷ −1)
By 1 Nov	Minimum system: CPU + RV32M runs a C test in simulation; VGA shows maze + tanks
Final phase	Integration: game.c with bullets, collision, health; timing closed; FPGA demo on Nexys A7

Dependencies: modules first, then pipeline integration, then game software, then bitstream

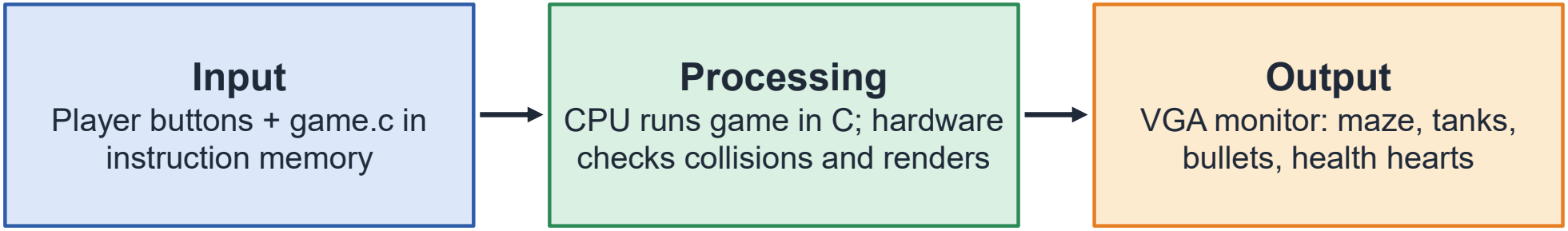
Team Responsibilities

Member	Responsibility
Kaustav	MUL / MULH / MULHSU / MULHU
Jayesh B	DIV / DIVU / REM / REMU + stall control
Kabir	Pipeline integration + hazards
Jayesh V	MMIO peripherals, VGA, game.c, FPGA build + demo
Mahendra	All testbenches + C programs to test

Interfaces: A/B → C: result + pipeline_stall; C ↔ D: MMIO register map

Demo Plan + Challenges

Demo Plan



Challenges

- **Stalls:** 32-clock divide must freeze pipeline, no double write
- **Signed/unsigned:** MULH variants, ÷0, INT_MIN ÷ −1 overflow
- **Hazards:** load wait + forwarding + branch squash together
- **FPGA timing:** 33×33 multiplier path at 50 MHz; VGA sync alignment

Backup: full system block diagram

